Séquence 2

Ouvrir et gérer une connexion WebSocket

2

Durant la séquence 1, vous avez démarré les deux services et expliqué le trajet d'un message. Aujourd'hui, vous allez brancher le navigateur au point **/chat**, afficher son état et fermer la connexion à la demande de l'utilisateur. Les raisons du choix de WebSocket sont acquises, nous passons à son cycle de vie.

1. Choisir la bonne adresse

Notre client doit viser :

```
ws://127.0.0.1:3000/chat
|        |        |        └── chemin de la gateway préparée
|        |        └─────────── port du backend
|        └──────────────────── machine locale
└───────────────────────────── WebSocket sans TLS, pour cet atelier local
```

Le port 5173 sert le frontend. Le port 3000 accueille le backend. **/health** est une route HTTP de diagnostic. **/chat** est le point d'entrée WebSocket. Ces trois destinations ne sont pas interchangeables.

Les protocoles WebSocket se déclinent en **ws://** et **wss://** (sécurisé via TLS). En production, un navigateur bloquera tout WebSocket non sécurisé sur une page HTTPS. Pour notre atelier local, nous utiliserons uniquement HTTP et **ws://**. Attention : remplacer manuellement **ws** par **wss** dans votre code ne suffit pas à activer le chiffrement, car notre serveur de développement n'est pas configuré pour gérer le TLS.

En apprendre + : Guide client WebSocket — MDN

2. Lire le handshake de notre atelier

Le navigateur crée la demande d'ouverture. Voici sa structure simplifiée pour HTTP/1.1. Les valeurs des clés changent selon la tentative.

```
GET /chat HTTP/1.1
Host: 127.0.0.1:3000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <valeur générée par le navigateur>
Sec-WebSocket-Version: 13
```

Une acceptation comporte notamment :

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: <valeur calculée par le serveur>
```

Le statut 101 indique l'acceptation du changement de protocole dans ce scénario HTTP/1.1. Une réponse HTTP 200 sur /health prouve autre chose : la route de diagnostic répond. Il ne faut pas traiter ces deux observations comme deux variantes d'un même succès.

Élément observé	Ce que vous devez en déduire
/chat, hôte et port	Le navigateur tente de joindre le bon point d'entrée
Upgrade: websocket	Il demande un changement de protocole
Sec-WebSocket-Version: 13	Version du protocole annoncée
Sec-WebSocket-Key et Sec-WebSocket-Accept	Éléments de vérification du handshake, pas un mot de passe utilisateur
101 Switching Protocols	Le serveur a accepté l'ouverture HTTP/1.1

Le navigateur génère la clé et vérifie la réponse. La clé n'authentifie pas une personne et ne chiffre pas les échanges. Vous ne calculez aucun en-tête dans votre code Vue : les implémentations du protocole s'en chargent.

Observer sans transformer la séance en laboratoire réseau

Lorsque votre client fonctionnera, ouvrez les outils de développement **avant** de cliquer sur Connecter. Dans Réseau/Network, filtrez les WebSockets si le navigateur le permet et sélectionnez l'URL terminant par **:3000/chat**. Relevez l'adresse et le statut de l'ouverture. La connexion de rechargement de Vite sur 5173 n'est pas la cible.

Une connexion peut rester indiquée comme active ou en attente dans la liste réseau : elle n'a pas vocation à se terminer juste après l'ouverture. Si vous ne trouvez pas les en-têtes, vérifiez d'abord l'URL sélectionnée.

3. Créer l'objet n'est pas attendre l'ouverture

```
const socket = new WebSocket('ws://127.0.0.1:3000/chat');
```

L'établissement d'une connexion WebSocket est asynchrone. Appeler **new WebSocket()** renvoie immédiatement l'objet et démarre la connexion en tâche de fond : il ne renvoie pas de Promesse et ne s'utilise pas avec await. Côté erreurs, il faut distinguer deux cas : une adresse mal formatée fera planter le code immédiatement, tandis qu'un serveur injoignable sera signalé plus tard, déclenchant les événements d'erreur sur l'objet WebSocket.

En apprendre + : Constructeur WebSocket — MDN

Dans l'exercice, le try/catch autour du constructeur est fourni. Il couvre le premier cas, pas toutes les pannes réseau. Pour le second, vous complétez les gestionnaires.

```
Clic Connecter
        |
        |— adresse invalide  →  exception immédiate  →  corriger l'adresse
        |
        └— objet créé  →  tentative en cours
                              |— ouverture acceptée  →  événement open
                              └— échec  →  événement error, puis close
```

Ce schéma sépare une erreur de construction et l'échec d'une tentative effectivement lancée. La durée avant le résultat dépend du réseau, ne déduisez pas une réussite du simple fait qu'aucune erreur n'est encore affichée.

`readyState : savoir où en est la connexion`

L'objet créé possède une propriété appelée readyState, que l'on peut comprendre comme « état de disponibilité ». Elle répond à la question : **« À cet instant, la connexion est-elle en cours d'ouverture, ouverte, en cours de fermeture ou fermée ? »**

On la lit avec **socket.readyState**, sans parenthèses : c'est une information portée par l'objet, pas une fonction à appeler. Sa valeur est un nombre mis à jour par le navigateur : **0** pendant la tentative, **1** lorsque la connexion est ouverte, **2** pendant la fermeture et **3** une fois fermée ou si l'ouverture a échoué. Le tableau de la section 5 reprend ces états et leurs conséquences pour l'interface.

```
const socket = new WebSocket('ws://127.0.0.1:3000/chat');
console.log(socket.readyState); // 0 : la tentative vient de commencer.

socket.addEventListener('open', () => {
  console.log(socket.readyState); // 1 : l'ouverture a réussi.
});
```

Cet exemple illustre la différence entre un état et un événement : **readyState** décrit la situation actuelle ; **open** nous prévient que l'ouverture vient de réussir. Lire la propriété une fois ne suffit pas à être informé de ses changements futurs. Ici, la fonction passée à **addEventListener** sera exécutée lorsque le navigateur signalera l'ouverture. Cet extrait sert à comprendre le mécanisme ; les écouteurs sont déjà fournis dans le TP.

Pour tester l'état, on utilise de préférence les constantes de l'API : **WebSocket.OPEN** est le nom donné à la valeur **1**.

```
if (socket.readyState === WebSocket.OPEN) {
  console.log('La connexion est ouverte.');
}
```

4. Relier événements et affichage

`open` : une ouverture réellement obtenue

L'événement open est le signal à utiliser pour afficher « Connecté ». Écrire ce texte juste après le constructeur produirait un faux succès si le serveur était arrêté.

En apprendre + : Evénement open — MDN

Le gestionnaire doit copier l'état courant de l'objet dans la donnée réactive de l'interface, puis renseigner le retour utilisateur.

`error` : un signal, pas un diagnostic complet

Dans le navigateur, error est un événement générique. Il ne fournit pas un diagnostic applicatif détaillé comme « mauvais port » ou « serveur éteint ». Le message affiché doit inviter à vérifier l'adresse, le serveur et les observations réseau, sans inventer la cause.

En apprendre + : Evénement error — MDN

Notre indicateur ne crée pas un cinquième état natif ERROR. Le texte de diagnostic et l'état de connexion sont deux informations distinctes. Une erreur de tentative sera suivie de la fermeture. C'est le gestionnaire close qui mettra à jour l'état final.

Une propriété technique ne rend pas Vue réactif à elle seule

Le navigateur modifie **socket.readyState**. Cette modification externe ne déclenche pas, à elle seule, une mise à jour d'une **ref** Vue. Le squelette fournit donc **readyState**, une donnée réactive qui reçoit l'état observé après construction, à l'ouverture, après la demande de fermeture et à la fermeture effective.

Les noms se ressemblent, mais il s'agit de deux données différentes : **current.readyState** est la propriété native de l'objet de cette tentative. **readyState.value** est notre copie destinée à Vue. Dans le script, **.value** permet d'accéder au contenu de la **ref**.

```
readyState.value = current.readyState;
```

Cette ligne lit le nombre sur l'objet et le copie dans la donnée réactive. Elle ne crée pas de liaison automatique : si l'on copie **0** à la construction, il faut refaire la copie dans le gestionnaire **open** pour que Vue reçoive ensuite **1**.

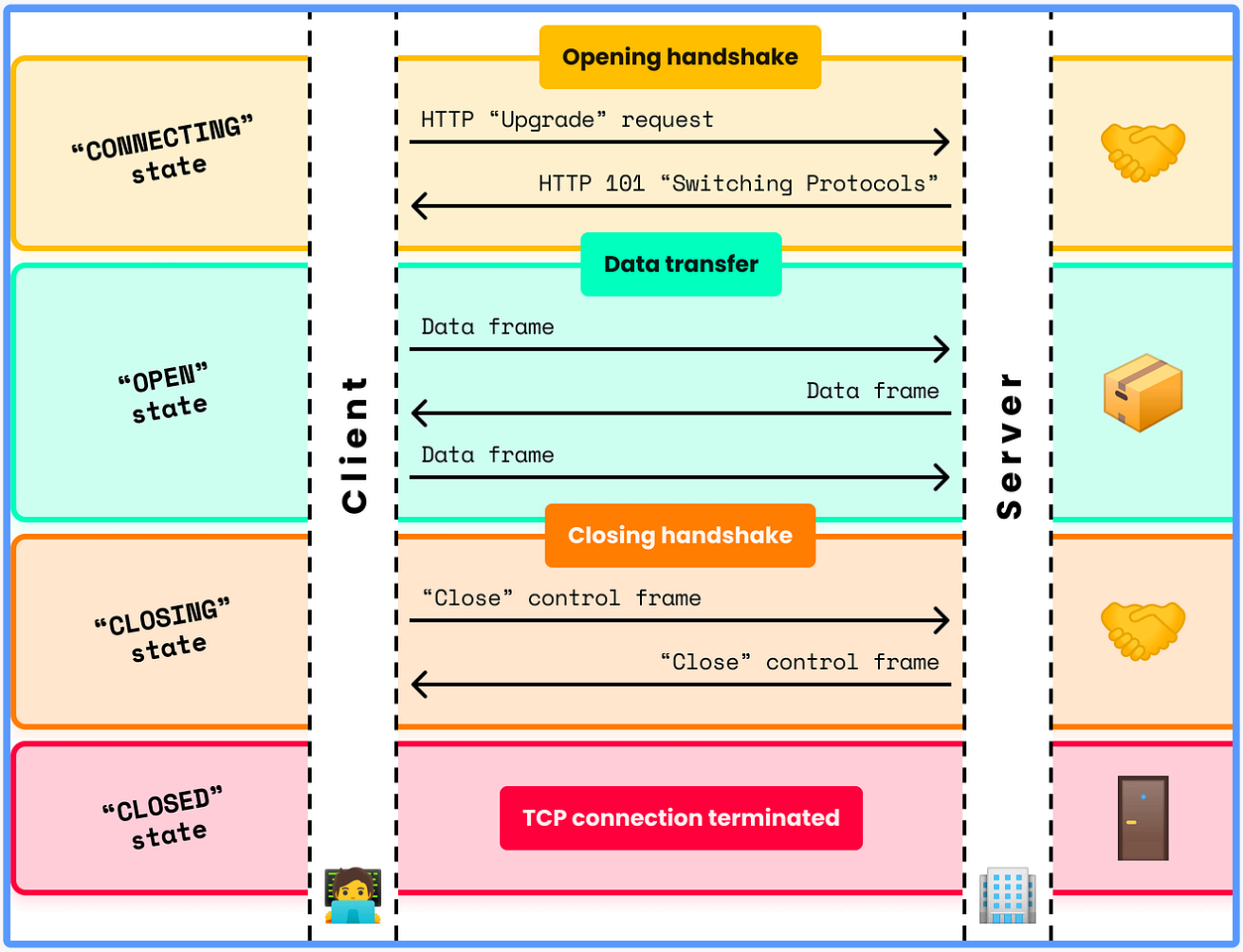
Cette copie évite un badge qui reste « Connexion en cours » alors que l'objet est déjà ouvert. Le problème de l'exercice est de synchroniser cette copie aux bons moments, pas de reconstruire le composant visuel.

5. Donner un sens aux états

Valeur native	Constante	Conséquence pour notre écran
0	Web Socket.CONNECTING	Tentative lancée. Ne pas en lancer une seconde
1	WebSocket.OPEN	Connexion établie. Autoriser la fermeture volontaire
2	WebSocket.CLOSING	Fermeture commencée. Attendre son résultat
3	WebSocket.CLOSED	Connexion fermée ou ouverture échouée. Autoriser une nouvelle tentative manuelle

null est seulement notre valeur d'interface pour « aucune tentative », avant la création d'un objet. Ce n'est pas un état WebSocket. Les constantes natives et leurs valeurs sont définies par l'API.

En apprendre + : Propriété readyState — MDN



Une interruption peut mener à **CLOSED** sans que l'écran ait le temps de montrer **CLOSING**. Le journal vous aide à distinguer les événements des transitions visuelles trop rapides. Il n'est pas nécessaire de ralentir artificiellement le réseau pour valider cette séquence.

6. Demander la fermeture, puis constater son résultat

Dans notre interface, Déconnecter est autorisé uniquement quand la connexion est ouverte :

```
socket.close(1000, 'Fin de la séance');
```

L'appel initie la fermeture ; il ne signifie pas que l'événement close a déjà été reçu. Le code 1000 exprime une fermeture normale. Après l'appel, l'interface peut indiquer « Fermeture en cours » puis afficher « Fermé » à réception de close. L'API autorise, pour un code fourni par ce client, 1000 ou une valeur applicative entre 3000 et 4999.

En apprendre + : Méthode close() — MDN

L'événement close donne notamment :

- code : code de fermeture observé .
- reason : raison textuelle, éventuellement vide.
- wasClean : indication de clôture propre de la connexion.

Ces données expliquent une fermeture, pas la livraison ou la lecture des messages métier.

Un code **1006** peut être rapporté localement lorsqu'aucune fermeture normale n'a été reçue. Il ne permet pas de conclure à une cause unique et ne doit pas être envoyé dans close(1006).

Une fois l'objet fermé, une nouvelle tentative utilise un nouvel objet. Ici, elle reste manuelle. Le programme de reconnexion automatique sera construit en S5.

7. Votre travail

Ouvrez le **document tp-guide**. Vous compléterez six emplacements dans frontend/src/useConnection.js. L'interface, le journal, le câblage des écouteurs et le nettoyage au démontage sont fournis. Vous ne modifiez pas le serveur.

Vous aurez terminé lorsque vous pourrez ouvrir, expliquer l'état affiché, fermer volontairement et diagnostiquer les cas demandés. L'étape suivante utilisera ce canal pour les messages. Elle ne fait pas partie du travail de cette séance.

Séquence 2

Terminée

Séquence 3 →